

UNIVERSIDAD ANÁHUAC MAYAB



División de Ingeniería y Tecnologías

Asignatura: Estructura de Datos

NAVEGADOR WEB

Integrantes del equipo:

- Dylan Herrera
- Andrés Elías
- Jonatan Fernando
- Antonio Julián Perdomo Peralta

Fecha de entrega: 20 de septiembre de 2026

Introducción

Este proyecto tiene el objetivo es modelar las operaciones básicas de navegación utilizando el concepto de **Pila** (*Stack*).

Para gestionar el historial de páginas se emplean dos pilas (*pila_atras* y *pila_adelante*) con un límite máximo de 4 elementos para controlar el uso de memoria. En los siguientes apartados se describen los requerimientos, el diseño, la estructura del código y la fase de pruebas para verificar el funcionamiento del programa y el manejo de los avisos de pila llena y vacía.

1. ¿Qué se debe hacer?

Desarrollar un programa en Python que simule el historial de un navegador web aplicando la estructura de datos Pila (LIFO). El sistema controla el flujo entre dos pilas fijas a un máximo de 4 elementos (*pila_atras* y *pila_adelante*) para permitir las operaciones de visitar páginas, retroceder (Back), avanzar (Forward) y consultar la página actual.

2. ¿Qué se necesita?

- Python 3 (el lenguaje obviamente)
- Listas dinámicas usadas como pilas mediante `append()` y `pop()`.
- Validaciones condicionales para detectar pila llena (X PILA LLENA) y pila vacía (X PILA VACIA).
- Menú interactivo en consola.

3. Elementos identificados

Elemento	Función dentro del sistema
Pila Atrás (<i>pila_atras</i>)	Guarda las páginas visitadas anteriormente (máximo 4 elementos).
Pila Adelante (<i>pila_adelante</i>)	Guarda las páginas que se desapilan al dar Back para permitir el Forward (máximo 4 elementos).

Página Actual (pagina_actual)	Almacena la URL en pantalla (about:blank al iniciar).
Capacidad Máxima (CAPACIDAD_MAXIMA)	Constante entera fija en 4.

Requerimientos para funcionar

Clave	Requerimiento
R1	Visitar página nueva
R2	Retroceder (Back)
R3	Avanzar (Forward)
R4	Ver página actual
R5	Control de límites (Pila Llena / Pila Vacía)

Detalles

R1 – Visitar página nueva

- **Descripción:** Guarda la pagina_actual en pila_atras (si no ha superado el tope de 4), actualiza la dirección activa con la nueva URL e inicializa pila_adelante en vacío.
- **Entrada:** Opción 1 del menú + URL.
- **Salida:** Visitando: <URL>

R2 – Retroceder (Back)

- **Descripción:** Desapila la última página de pila_atras y la define como la activa. La página actual se guarda en pila_adelante.
- **Entrada:** Opción 2 del menú.
- **Salida:** regresaste a: <URL> o X PILA VACIA si no hay elementos.

R3 – Avanzar (Forward)

- **Descripción:** Desapila la última página de pila_adelante y la define como la activa. La página actual se guarda en pila_atras.
- **Entrada:** Opción 3 del menú.
- **Salida:** avanzaste a: <URL> o X PILA VACIA si no hay elementos.

R4 – Ver página actual

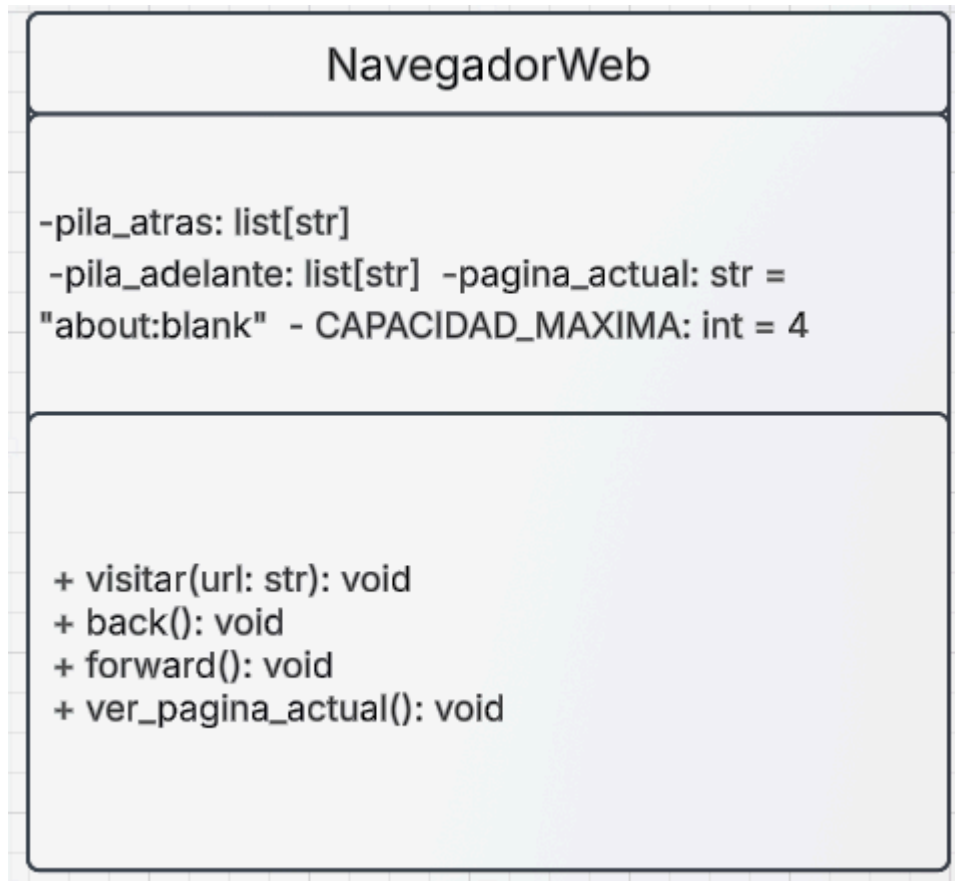
- **Descripción:** Despliega la URL que está cargada en ese momento.
- **Entrada:** Opción 4 del menú.
- **Salida:** estas en: <URL>

R5 – Manejo de desbordamiento y subdesbordamiento

- **Descripción:** Imprime X PILA LLENA cuando se intenta agregar un elemento y la pila alcanzó los 4 registros. Imprime X PILA VACIA al intentar un desapilado sobre una pila sin elementos.
- **Entrada:** Operaciones en los límites de la Pila.
- **Salida:** X PILA LLENA / X PILA VACIA

Diseño

El sistema se organiza alrededor de la clase NavegadorWeb. Sus atributos son privados para mantener el estado de las pilas y las operaciones del historial se exponen mediante métodos públicos. La clase administra el control de las páginas y la salida en consola asociada a cada acción.



(Diagrama UML)

Atributos y su proposito.

Atributo	Propósito
pila_atras	Lista dinámica que almacena las páginas pasadas (máximo 4 cadenas).
pila_adelante	Lista dinámica que almacena las páginas desaparecidas tras usar Back (máximo 4 cadenas).
pagina_actual	Cadena que guarda la URL activa en pantalla (about:blank por defecto).
CAPACIDAD_MAXIMA	Entero con valor fijo de 4; determina el tope máximo de las pilas.

El constructor `__init__` inicializa las listas `pila_atras` y `pila_adelante` como listas vacías, asigna "about:blank" a `pagina_actual` y fija `CAPACIDAD_MAXIMA` en 4. Los métodos `visitar`, `back`, `forward` y `ver_estado` contienen la lógica de cada opción del menú. El programa se estructura en una sola clase dentro de un mismo archivo, por lo que no hace falta usar herencia.

Fuera de la clase se ejecuta un ciclo `while` que despliega el menú en consola, lee la opción que escribe el usuario y llama al método que corresponde usando el objeto `navegador`.

Codigo

El código se desarrolló en Python utilizando un único archivo ejecutable. Se usan las operaciones nativas de listas (`append`, `pop`, `clear`) para manejar las pilas bajo la lógica LIFO y un bloque `try-except` con `input().strip()` para validar la entrada en consola.

Inserción y control de límite en la Pila

En el método `visitar()`, antes de guardar una página en `pila_atras`, se revisa si ya se alcanzó la `CAPACIDAD_MAXIMA` de 4 elementos. Si no se ha llenado, se guarda la página actual en la pila antes de cambiar a la nueva URL:

Retroceso, avance y desapilado LIFO

En los métodos `back()` y `forward()` se valida primero que la pila no esté vacía (`len == 0`). Para recuperar el último sitio visitado se usa `pop()`, lo que saca el elemento más reciente respetando la estructura LIFO

Opción	Operación
1	<code>navegador.visitar(u)</code>
2	<code>navegador.back()</code>

3	<code>navegador.forward()</code>
4	<code>navegador.ver_estado()</code>
5	Salir del programa

Un ciclo `while` mantiene el menú activo y la estructura condicional `if-elif-else` dirige cada opción seleccionada por el usuario. La validación de la entrada se realiza mediante un bloque `try-except` que captura errores de conversión de tipo (`ValueError`), verifica que no se ingresen cadenas vacías ni texto no numérico, y despliega el mensaje "opcion invalida" en caso de detectar una falla sin detener la ejecución.

El código incluye comentarios explicativos en las variables iniciales, los métodos de la clase `NavegadorWeb`, el constructor y la estructura del menú principal para detallar el flujo y la lógica de cada operación.

Ejecución del entorno

Para la ejecución en entorno local o plataformas en línea como OnlineGDB, se selecciona Python 3 como lenguaje, se pega el script en el editor principal y se ejecuta mediante la consola interactiva (`Interactive Console`). El programa no requiere la instalación de librerías externas ni dependencias adicionales para su funcionamiento.

Pruebas del funcionamiento

Se ejecutó el código `NavegadorWeb` utilizando Python 3 en [Onlinegdb.com](https://www.onlinegdb.com/). Se comprobaron los resultados obtenidos contra el comportamiento esperado en cada caso. Las pruebas abarcan la navegación estándar, el control de límites de capacidad (4 elementos) en las pilas y la validación de entradas inválidas en el menú.

Caso	Req.	Acción y resultado esperado	Resultado observado
P1	R4 / R5	Consultar página al iniciar sin navegar.	Muestra estas en: about:blank. Correcto.
P2	R1	Visitar cuatro páginas seguidas: perro.com, gato.com, perritos.com y garfield.com.	Agregó las URLs a pila_atras e imprimió la confirmación de visita en cada una. Correcto.
P3	R1 / R5	Intentar visitar una quinta página (donatello.com) teniendo la pila llena.	Desplegó X PILA LLENA y cambió la página activa a donatello.com. Correcto.
P4	R2	Ejecutar Back desde la quinta página.	Regresó a perritos.com y guardó donatello.com en pila_adelante. Correcto.
P5	R3	Ejecutar Forward tras el retroceso.	Avanzó de nuevo a donatello.com. Correcto.
P6	R2	Retroceder consecutivamente hasta agotar el historial de pila_atras.	Pasó por perritos.com, gato.com, perro.com hasta llegar a about:blank. Correcto.

P7	R2 / R5	Intentar retroceder (Back) cuando pila_atras está vacía.	Desplegó X PILA VACIA y no modificó el estado. Correcto.
P8	N/A	Ingresar letras, texto no numérico o valores fuera del rango (1-5).	Desplegó opcion invalida y volvió a pedir una opción. Correcto.
P9	N/A	Presionar Enter sin escribir texto o enviar una entrada vacía.	Finalizó la ejecución del programa mostrando saliendo.... Correcto.

Evidencias.

La capacidad máxima de la pila de atrás se alcanzó con cuatro direcciones URL visitadas (perro.com, gato.com, perritos.com, garfield.com). Al intentar agregar una quinta página (donatello.com), el programa desplegó el aviso de pila llena, asignó la nueva URL a la página activa y descartó la inserción en el historial acumulado.

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 1
url: perro.com
Visitando: perro.com
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 1
url: gatos.com
Visitando: gatos.com
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 1
url: perritos.com
Visitando: perritos.com
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 1
url: garfield.com
Visitando: garfield.com
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 1
url: donatello.com
X PILA LLENA
Visitando: donatello.com
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: █
```

(Se puede observar el funcionamiento de pila al llenarse)

vaciado de pila y desbordamiento inferior

Posteriormente, se ejecutaron operaciones de retroceso (Back) consecutivas hasta vaciar la pila_atras y regresar a la página inicial (about :blank). Al intentar realizar operaciones adicionales de retroceso o avance sin registros disponibles en el historial, el programa desplegó el mensaje correspondiente sin provocar fallos de ejecución.

(Foto mas abajo)

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 2
regresaste a: perritos.com
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 2
regresaste a: gatos.com
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 2
regresaste a: perro.com
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 2
regresaste a: about:blank
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 2
X PILA VACIA
```

```
1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: █
```

(Se puede ver el sistema de vaciamiento de pila)

Ver pagina actual, back y forward.

```
4 - Ver pagina actual
5 - Salir
opcion: 1
url: tortugas.com
Visitando: tortugas.com

1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 4
estas en: tortugas.com

1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 1
url: pequenocesar.com
Visitando: pequenocesar.com

1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 2
regresaste a: tortugas.com

1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: 3
avanzaste a: pequenocesar.com

1 - Visitar pagina nueva
2 - Back
3 - Forward
4 - Ver pagina actual
5 - Salir
opcion: █
```

(Se puede ver el sistema de de back, forward y ver pagina.)

Conclusiones.

Con este proyecto se comprobó cómo funciona en la práctica una pila (LIFO) para controlar el historial de un navegador web. Separar la lógica en dos pilas (pila_atras y pila_adelante) fue una forma sencilla y directa de manejar las opciones de regresar y avanzar de página sin perder el orden de los sitios visitados.

También se validó el control de los límites en memoria fijando un tope de 4 elementos. Esto evitó errores al intentar desapilar cuando no hay datos (X PILA VACIA) o al querer guardar más de lo permitido (X PILA LLENA). Al final, organizar el código en la clase NavegadorWeb permitió mantener el estado de las pilas aislado del menú que interactúa con el usuario.

Referencias usadas

Contenido en brightspace de la clase

[202660_50560_Estructura_De_Datos_Y_ALGORITMOS](#)

https://www.youtube.com/watch?v=9_IOdb9ELRU